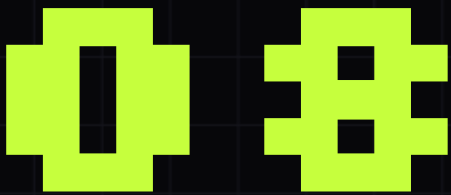


MODULE 08 OF 09



Monetization

YOU'LL SHIP A working in-game store.

- 8.1 Game Passes
- 8.2 Developer Products
- 8.3 Bulletproof ProcessReceipt
- 8.4 Pricing that doesn't push players away

Roblox games earn Robux through two kinds of purchases: **Game Passes** (buy once, own forever) and **Developer Products** (buy as many times as you like). In this module you'll add both to Coin Rush, handle purchases without ever losing a player's Robux, and price them so players feel good about paying.

8.1 Game Passes

A Game Pass is a permanent perk. For Coin Rush: **2× Coins**.

Create it:

1. Open create.roblox.com → **Creations**, select your experience.
2. Go to **Monetization** → **Passes** → **Create a Pass**, and upload an icon (512×512).
3. After creating it, open the pass, set it **For Sale** and choose a price.
4. Copy the **Pass ID** (the number in the URL or on the pass page).

Check ownership and apply the perk (server):

```
local MarketplaceService = game:GetService("MarketplaceService")
local Players = game:GetService("Players")

local DOUBLE_COINS_PASS = 0000000000 -- your Pass ID

local function checkPass(player: Player)
    local ok, owns = pcall(function()
        return MarketplaceService:UserOwnsGamePassAsync(player.UserId, DOUBLE_COINS_PASS)
    end)
    player:SetAttribute("DoubleCoins", ok and owns)
end

Players.PlayerAdded:Connect(checkPass)

-- When they buy it during the session
MarketplaceService.PromptGamePassPurchaseFinished:Connect(function(player, passId,
    purchased)
    if purchased and passId == DOUBLE_COINS_PASS then
        player:SetAttribute("DoubleCoins", true)
    end
end)
```

Then in `PlayerDataService.addCoins`, multiply when the attribute is set:

```
if player:GetAttribute("DoubleCoins") then
    amount *= 2
end
```

Prompt the purchase (client):

```
local MarketplaceService = game:GetService("MarketplaceService")
local player = game:GetService("Players").LocalPlayer

buyDoubleCoinsButton.Activated:Connect(function()
    MarketplaceService:PromptGamePassPurchase(player, DOUBLE_COINS_PASS)
end)
```

#.2 Developer Products

Developer Products can be bought repeatedly: coin packs, revives, skip-a-wait. For Coin Rush: **500 Coins**.

Create it under **Monetization** → **Developer Products** → **Create a Developer Product**, set the price, and copy the **Product ID**.

Prompt it from the client:

```
MarketplaceService:PromptProductPurchase(player, COIN_PACK_500)
```

The actual reward is given by the server in `ProcessReceipt`, the most important purchase function in Roblox.

#.3 Bulletproof ProcessReceipt

When a player buys a Developer Product, Roblox calls your `MarketplaceService.ProcessReceipt` callback on the server. You grant the item and return `PurchaseGranted`. If you return `NotProcessedYet` (or the server crashes), Roblox **tries again later**, even in another server. That retry is what protects players' Robux.

Rules:

1. Set `ProcessReceipt` **once**, in **one** server Script.
2. Only return `PurchaseGranted` **after** the reward is actually given and saved.
3. Remember which receipts you already granted, so a retry doesn't give the reward twice.

```

-- ServerScriptService/Purchases
local MarketplaceService = game:GetService("MarketplaceService")
local Players = game:GetService("Players")
local DataStoreService = game:GetService("DataStoreService")
local ServerScriptService = game:GetService("ServerScriptService")

local PlayerDataService = require(ServerScriptService.PlayerDataService)
local receipts = DataStoreService:GetDataStore("PurchaseReceipts_v1")

local COIN_PACK_500 = 0000000000 -- your Product ID

local handlers: { [number]: (Player) -> boolean } = {
    [COIN_PACK_500] = function(player)
        PlayerDataService.addCoins(player, 500, { ignoreMultiplier = true })
        return true
    end,
}

MarketplaceService.ProcessReceipt = function(receipt)
    local key = `{receipt.PlayerId}_{receipt.PurchaseId}`

    -- Already granted? Tell Roblox we're done.
    local ok, alreadyGranted = pcall(function()
        return receipts:GetAsync(key)
    end)
    if not ok then
        return Enum.ProductPurchaseDecision.NotProcessedYet
    end
    if alreadyGranted then
        return Enum.ProductPurchaseDecision.PurchaseGranted
    end

    -- The player must be in this server to receive the item.
    local player = Players:GetPlayerByUserId(receipt.PlayerId)
    local handler = handlers[receipt.ProductId]
    if not player or not handler then
        return Enum.ProductPurchaseDecision.NotProcessedYet
    end

    local granted = handler(player)
    if not granted then
        return Enum.ProductPurchaseDecision.NotProcessedYet
    end
end

```

```

-- Record the receipt, then save the player's data right away.
local saved = pcall(function()
    receipts:SetAsync(key, true)
end)
if not saved then
    return Enum.ProductPurchaseDecision.NotProcessedYet
end
PlayerDataService.saveNow(player)

return Enum.ProductPurchaseDecision.PurchaseGranted
end

```

Two small additions to `PlayerDataService` make this work: expose the `save` function from Module 6 as `saveNow`, and give `addCoins` an optional `{ ignoreMultiplier = true }` flag so bought coins aren't doubled by the 2× pass.

WATCH OUT Never grant developer products from a `RemoteEvent` or from `PromptProductPurchaseFinished`. Those can fire even when no Robux were paid. `ProcessReceipt` is the only trustworthy signal.

Testing: purchases in Studio are simulated and free. Test every product in Studio, then once more in the live game with a cheap price.

Pricing that doesn't push players away

Money follows goodwill. Players pay when the game already feels generous.

Principles:

- **Fun first, for free.** A free player must be able to enjoy and finish everything. Paid items make it *faster, cooler* or *more convenient*, not possible.
- **Avoid pay-to-win in competitive modes.** In Coin Rush, 2× Coins speeds up progression but doesn't give an edge inside a round.
- **Offer a cheap first purchase.** A 25–50 Robux item gets players over the "first purchase" barrier.
- **Anchor with a bigger bundle.** Show the 500-coin pack next to a 1,500-coin pack with a "best value" label.
- **Show, don't nag.** Place the shop button clearly; never block gameplay with repeated purchase pop-ups.
- **Time offers to moments of desire**, like after a close loss ("Revive for 15 R\$") or when a player has almost enough coins for an upgrade.

Typical price ranges (always test for your game): cosmetic items 25–150 R\$, 2× passes 99–249 R\$, VIP pass 149–499 R\$, currency packs from 25 R\$ upward.

Track results in **Analytics** → **Monetization** (Module 9) and change one price at a time so you know what caused a change.

TRY IT

1. Create the 2× Coins pass and the 500 Coins product, and test both in Studio.
2. Add a "best value" 1,500-coin product and see which one testers pick.

Module recap

- ☐ 2× Coins Game Pass works, including when bought mid-session
- ☐ 500 Coins Developer Product granted only in ProcessReceipt
- ☐ Receipts are recorded, so retries never double-grant
- ☐ Store prices follow the fun-first rules